Layer 2 (Scores) - Research Document

Filip Stachura, with Claude

2026-06-12

CONTENTS

Layer 2 (Scores) — Research Document

Date: 2026-06-12 **Author:** Filip Stachura, with Claude **Status:** Research — no code, no PRs **Builds on:** ADR-0007 (Layer 1 shipped in PR #677)

---

1. User Workflow Integration

KEY DESIGN QUESTION

How do Scores fit into the daily work of three personas — workflow operators, agent developers, compliance/QA — without adding friction or duplicating existing surfaces?

1A. WORKFLOW OPERATORS (PHARMA TEAMS RUNNING WORKFLOWS)

When they see scores. Operators already interact with agent results at two touchpoints: (1) the run step detail view (showing the Agent Output Envelope — result, confidence, reasoning), and (2) the L3 human review flow where they approve/reject/revise agent output. Scores should appear alongside these, not in a separate "evaluation" area.

Human verdict = ground truth Score. The L3 approval flow (`complete-human-task.ts` → `buildVerdictStepOutput()`) already captures a structured verdict: `{key, intent, comment}`. This IS the highest-quality Score source — a domain expert's judgment on agent output. The platform should auto-create a Score when a HumanTask with `creationReason: 'agent_review_l3'` completes, with zero extra operator action:

```
Score {
  subject:    { type: 'agent_run', id: agentRunId }
  name:       'human_verdict'
  source:     'human'
  value:      intent === 'success' ? 1.0 : intent === 'danger' ? 0.0 : 0.5
  label:      verdict.key          // e.g. 'approve', 'reject', 'revise'
  comment:    verdict.comment      // operator's free-text reasoning
  createdBy:  userId
  agentRunId, processInstanceId, namespace, stepId  // correlation
}
```

This bridges the existing workflow surface to the evaluation domain with no new UI and no new operator behavior. The `intent` → numeric mapping is lossy on purpose — it normalizes across different verdict vocabularies (workflows define custom verdicts) for aggregation. The `label` preserves the original verdict key.

Deterministic scores on every agent run. Some checks need no human or LLM — they can fire automatically after every agent step completes:

- **Has result**: `envelope.result !== null` (binary)
- **Confidence threshold**: `envelope.confidence >= step.agent.minConfidence`
- **Token budget**: `envelope.tokenUsage.total <= step.agent.maxTokens`
- **Duration**: `envelope.duration_ms <= step.agent.timeoutMs`
- **Schema compliance**: does `envelope.result` match the step's expected output schema (if one is defined)?

These are cheap, instant, and provide the baseline quality signal even for L0/L1 autonomous runs where no human reviews the output.

1B. AGENT DEVELOPERS (BUILDING/TUNING AGENT PLUGINS)

The feedback loop. Today an agent developer can see individual run results in the UI and traces in Phoenix. What's missing: systematic comparison across runs. The Score entity enables:

1. **Run agent** → Scores auto-generated (deterministic + optional LLM judge)
2. **See scores** → per-run detail + aggregated per-agent view
3. **Change prompt/model** → re-run same workflow step
4. **Compare** → side-by-side scores: old model vs new, old prompt vs new

The comparison surface is the precursor to Layer 4 (Eval Runs / champion vs challenger), but simple aggregation ("average human_verdict score for agent X over the last 30 days") is Layer 2 work.

LLM-as-judge for developer iteration. `ReviewPlugin.review()` already returns `{verdict: ReviewVerdict, reasoning: string, feedback?: string, confidence: number}` — this is an LLM-as-judge interface that exists but isn't wired to scoring. Mapping:

```
ReviewPluginResult → Score {
  name:       'llm_judge'
  source:     'llm_judge'
  value:      verdict === 'approve' ? 1.0 : verdict === 'reject' ? 0.0 : 0.5
  label:      verdict                       // 'approve' | 'reject' | 'revise'
  comment:    reasoning
  metadata:   { judgeModel, confidence, feedback }
}
```

The developer can attach LLM-as-judge scoring to a workflow step configuration: "after the agent runs, also run ReviewPlugin and store its output as a Score." This is opt-in per step — not every run needs an LLM judge (cost, latency).

1C. COMPLIANCE/QA TEAMS

Scores as audit trail. Every Score write MUST produce an AuditEvent via the existing `AuditRepository.append()`. The AuditEvent captures:

- `actorType`: 'user' (human verdict), 'agent' (LLM judge), 'system' (deterministic check)
- `action`: 'score.created'
- `entityType`: 'score', `entityId`: scoreId
- `inputSnapshot`: the scoring input (agent output being judged)
- `outputSnapshot`: the Score value + reasoning
- `basis`: which scoring rule/config triggered this

This means the audit trail already knows WHO scored WHAT, WHEN, and WHY — the GxP qualification requirement from ADR-0007 D2.

Immutability. Scores should be append-only. A revised judgment creates a new Score with a `supersedes` pointer, not an update. This matches AuditEvent's append-only pattern and 21 CFR Part 11 requirements.

Model swap justification. "Model M passed eval suite S before deployment into workflow W" = a query across Scores: "all Scores where subject is an AgentRun using model M in workflow W, grouped by Score name, with passing averages." This is the Layer 4 (Eval Run) story, but the Score entity must carry enough correlation data to make the query possible at Layer 2.

1D. DASHBOARD / REPORTING

Aggregations that matter:

DIMENSION	QUESTION ANSWERED
Per agent	"Is agent X getting better or worse?"
Per model	"Is claude-sonnet-4 good enough to replace opus?"
Per workflow	"Which workflow has the lowest quality?"
Per step	"Which step in this workflow fails most?"
Per scorer	"Which scoring criteria fail most?"
Over time	"Quality trend for agent X over last 30 days"

Drift detection = score trend going down. Simplest viable: compute a rolling average of Score values per (agent, scorer name) over time. Alert when the 7-day average drops below the 30-day average by >N%. This is purely a read concern — no new entity needed beyond Score. The complexity is in the aggregation query and the alerting threshold, not the data model.

Dashboard scope. Layer 2 delivers: per-run score detail (on the existing run/step detail pages), per-agent score summary (new lightweight view or card), and a time-series chart of scores over time. Full dashboarding with custom queries and drill-downs is Layer 3-4 territory.

---

2. External Tools Landscape

KEY DESIGN QUESTION

What's worth using vs building? ADR-0007 D2 says Scores live in-platform. But external tools have scoring UIs, eval runners, annotation queues. Where exactly is the line?

2A. PHOENIX (ALREADY DEPLOYED)

What it has:

- `px.Client().log_evaluations(SpanEvaluations(eval_name, dataframe))` — pushes scores to Phoenix keyed by `span_id`, displayed alongside traces
- `TraceEvaluations` for trace-level scores
- Experiments API: `run_experiment(dataset, task, evaluators)` with built-in evaluators (Hallucination, QA, Relevance, Toxicity, Summarization) and code evaluators (ContainsKeyword, MatchesRegex, etc.)
- `arize-phoenix-evals` Python package: `llm_classify`, `run_evals`, model wrappers (OpenAI, Anthropic, Bedrock, LiteLLM)
- Dataset upload/management via `px.Client().upload_dataset()`

Architecture detail: Phoenix stores annotations in a **separate table** (`span_annotations`), not inside OTel span attributes. Annotations are a mutable overlay on immutable OTel data — linked by `span_id` as FK. This is the right model: spans are portable to any backend, scores are Phoenix-specific. Upsert semantics on `(name, span_id, identifier)`.

What we should use: Score visualization alongside traces. When a Score is created in-platform, sync it to Phoenix via `POST /v1/span_annotations` (JSON REST endpoint, works from TypeScript via `@arizeai/phoenix-client`). Write-only, platform → Phoenix.

```
// TypeScript — push a score to Phoenix
await phoenix.POST("/v1/span_annotations", {
  body: { data: [{
    span_id: agentRun.otelSpanId,
    name: score.name,
    annotator_kind: score.source === 'human' ? 'HUMAN'
                  : score.source === 'llm_judge' ? 'LLM' : 'CODE',
    result: { label: score.label, score: score.value,
              explanation: score.comment },
    metadata: score.metadata ?? {},
  }]}
});
```

What we should NOT use: Phoenix's Experiments/Datasets as the system of record. ADR-0007 D2 is clear: eval entities are platform-owned for namespace scoping, audit events, and GxP qualification. Phoenix datasets have no namespace concept, no audit trail, no authz model.

What we should watch: Phoenix's `arize-phoenix-evals` evaluator library. The built-in evaluators (HallucinationEvaluator, QAEvaluator) are well-tested and could be used as judge implementations invoked by our platform — but the dependency is Python-only. For a TypeScript platform, we'd reimplement the prompt templates in our own judge pipeline (the prompts are open source, the pattern is simple classification with rails).

Drift detection caveat: Phoenix OSS has NO score-over-time drift detection or alerting. Only embedding drift visualization (UMAP/HDBSCAN). Score time-series charts exist in the UI but no automated thresholds. Our platform must own drift detection — a simple rolling-average comparison is Layer 2 scope; statistical process control is later.

Recommendation: USE for visualization, IGNORE for system of record, WATCH eval library prompts.

2B. LANGFUSE

What it has:

- Scores: `POST /api/public/scores` — rich schema with `dataType` (NUMERIC, CATEGORICAL, BOOLEAN, TEXT), `configId` for constraints, `source` (API | EVAL | ANNOTATION)
- OTLP ingestion at `/api/public/otel` — scores attach to OTLP-ingested traces via the same API using hex trace_id
- Annotation queues with assignees, score configs

- Datasets: `input` / `expectedOutput` / `metadata` with optional JSON Schema
- Eval runner: `langfuse.experiment(dataset, task, evaluators)`

Should we reconsider ADR-0007's "don't adopt Langfuse SDK"?

No. The reasoning holds:

1. **Namespace scoping**: Langfuse has "projects" but no equivalent to our namespace → workspace → membership model. Scores in Langfuse can't inherit platform authz.
2. **Audit Events**: Langfuse has no append-only audit log meeting 21 CFR Part 11. Score writes in Langfuse are unaudited.
3. **GxP qualification**: A second system doubles qualification scope.
4. **Self-host footprint**: Langfuse v3 = ClickHouse + Redis + S3 + Postgres — heavy for single-tenant on-prem.
5. **Invariant enforcement**: Platform can't gate model swaps on Langfuse data it doesn't own.

However: Langfuse remains a valid **OTLP target** per ADR-0007 D3. A customer who already runs Langfuse can point the OTel Collector at it and get trace visualization. If we sync Scores to Phoenix, the same mechanism could optionally sync to Langfuse via its scores API. But this is a nice-to-have, not Layer 2 scope.

Recommendation: IGNORE for scoring system of record. INTEGRATE as optional OTLP trace target (already supported by D3). WATCH annotation queue UX for design inspiration.

2C. BRAINTRUST

What it has:

- Experiments with `Record<string, number>` scores (0-1 only, no categorical)
- `autoevals` library: `Factuality`, `ClosedQA`, `LLMClassifierFromTemplate` — discrete classification mapped to numeric via `choiceScores`, reducing variance
- Datasets with automatic versioning (`_xact_id`), experiments pin to dataset version
- Feedback endpoint for updating scores post-hoc
- TypeScript-native SDK

What's worth borrowing:

- The `Score` interface: `{name: string, score: number, metadata?: Record<string, unknown>}` — simple, composable. Our Score schema should be equally simple at its core.
- `LLMClassifierFromTemplate` pattern: define discrete choices mapped to numeric scores, force LLM to classify. More reliable than asking for a raw 1-10 number. Our LLM-as-judge implementation should use this pattern.
- Experiment-dataset version pinning: when we build Layer 3-4, snapshot the dataset version used in each eval run.

Recommendation: IGNORE as platform (cloud-only, no self-host). BORROW patterns for Score schema design and LLM-as-judge reliability.

2D. HUMANLOOP

Acquired by Anthropic in 2025; standalone platform sunset September 2025. Not a viable integration target.

Worth noting: Their evaluator taxonomy (Code / AI / Human) maps exactly to our three Score sources (deterministic / LLM-as-judge / human review). And their "online evaluators" concept (auto-run on production logs for drift detection) is worth considering for our deterministic scores.

Recommendation: IGNORE. BORROW evaluator taxonomy naming.

2E. LLM-AS-JUDGE FRAMEWORKS

FRAMEWORK	LANGUAGE	KEY PATTERN	RELEVANCE
RAGAS	Python	`AspectCritic(definition, strictness)` — multi-vote	Medium
DeepEval	Python	`GEval(criteria, evaluation_steps)` — CoT + 0-10 scale	Medium
promptfoo	TypeScript	YAML `llm-rubric` assertions, `GradingResult`	High
autoevals	TS+Python	`LLMClassifier(choiceScores)` — discrete classification	High
LangSmith	Python	`CriteriaEvalChain`, `EvaluationResult(key, score)`	Low

Cross-framework patterns we should adopt:

1. **Rubric = free-text string.** Every framework accepts plain English. Our `ScoreConfig` should have a `rubric: string` field.
2. **Score normalization to 0-1 float.** Universal. No reason to diverge.
3. **Multi-criteria = composition.** N independent scorers, not one multi-output scorer. Each Score is one (name, value) pair.
4. **Score + reason.** Always capture reasoning alongside the numeric value. Essential for debugging and audit.
5. **CoT before scoring.** Reasoning field MUST precede score in the LLM schema to force chain-of-thought. +10-20% consistency.
6. **Discrete classification > raw numbers.** Binary/categorical choices mapped to numeric are more reliable than asking an LLM for a 1-10 rating. Use the `choiceScores` pattern.

Recommendation for LLM-as-judge implementation: Don't adopt an external framework as a runtime dependency. Our platform already has `ReviewPlugin` with the right interface shape and `OpenRouterLlmClient` for LLM calls. Build the judge pipeline in TypeScript using patterns from autoevals (discrete classification, CoT). The judge prompt templates can be platform-configurable (Score Config entity, Layer 2 scope) or hardcoded initially.

2F. SUMMARY TABLE

TOOL	VERDICT	REASON
Phoenix	**USE**	Trace viz + score overlay via `POST /v1/span_annotations`
Langfuse	INTEGRATE	Optional OTLP target per D3; don't adopt scoring
Braintrust	IGNORE	Cloud-only; borrow Score schema + classifier pattern
Humanloop	IGNORE	Sunset; borrow evaluator taxonomy
RAGAS	IGNORE	Python-only; borrow multi-vote pattern if needed
DeepEval	IGNORE	Python-only; borrow CoT + evaluation_steps pattern
promptfoo	**WATCH**	TypeScript, good assertion model; possible future CLI tool
autoevals	IGNORE	Borrow `LLMClassifier` + `choiceScores` pattern
LangSmith	IGNORE	LangChain ecosystem; not relevant to our stack

3. Integration with Layer 1 (Traces)

KEY DESIGN QUESTION

How do Scores connect back to trace data when traces live externally (Phoenix/Jaeger/Datadog) and Scores live in-platform?

3A. THE JOIN KEY: AGENTRUNID, NOT TRACEID

The correlation contract (D4) puts `mediforce.agent_run.id` on every span. Score.subject is `{type: 'agent_run', id: agentRunId}`. This means:

- **Platform → trace store**: Given a Score, find its traces by querying the trace store for spans where `mediforce.agent_run.id = score.subjectId`. This works regardless of which trace backend is deployed.
- **Trace store → platform**: Given a trace, find its Scores by querying the platform for Scores where `subjectId = span.attributes['mediforce.agent_run.id']`.

The join is `agentRunId`, a platform-owned ID. We do NOT use traceId or spanId as the join key because: (a) they're generated by OTel, not the platform; (b) they're ephemeral — trace stores have retention policies; (c) traceId format varies by backend (hex vs b64). The platform owns the identity; the trace store is a view.

For Workflow Run-level Scores, the join key is `processInstanceId`. Same logic — the platform ID is canonical.

3B. PUSHING SCORES TO PHOENIX FOR VISUALIZATION

Mechanism: After a Score is created in-platform, push it to Phoenix via `POST /v1/span_annotations` (REST, JSON, works from TypeScript — see §2a code snippet). This requires knowing the `span_id` of the root agent run span.

Problem: The platform doesn't currently store the OTel span ID. The span is created by `withAgentRunSpan()` in `agent-runner.ts` and exported to the trace backend, but the span ID isn't persisted on the AgentRun entity.

Options:

OPTION	DESCRIPTION	TRADEOFF
A. Store spanId on AgentRun	Add `otelSpanId?: string` to AgentRunSchema, set it in `withAgentRunSpan()`	Clean join. Small schema change.
B. Query Phoenix by agentRunId attribute	Phoenix supports querying spans by attribute value	Works, but couples the sync path to Phoenix's query API
C. Don't sync to Phoenix	Scores visible only in platform UI	Misses the "see quality alongside traces" benefit

Recommendation: Option A. Store the OTel span ID on the AgentRun at creation time. It's a single field addition, set in the same code path that already creates the span. Then the Phoenix sync is a simple lookup: `agentRun.otelSpanId` → `SpanEvaluations(span_id=otelSpanId)`.

Sync direction: Platform → Phoenix only. Never read Scores back from Phoenix. The platform is the system of record (D2). Phoenix is a visualization convenience. If Phoenix is unavailable, Scores still exist.

Sync trigger: Background job or event hook after Score creation. Not synchronous — Score creation should not block on Phoenix availability. Initially, a simple async fire-and-forget; later, a proper outbox pattern if reliability matters.

3C. CONTENT ACCESS FOR LLM-AS-JUDGE SCORING

LLM-as-judge needs the prompt and completion to evaluate. Where does it get them?

SOURCE	PROS	CONS
AgentOutputEnvelope (in-platform)	Always available. Has result, reasoning_chain, confidence. No external dependency.	Doesn't have the raw prompt (system prompt + user message). Has the output but not the exact LLM conversation.
Trace store (Phoenix)	Has full prompt/completion when `MEDIFORCE_OTEL_CAPTURE_CONTENT=true`.	External dependency. Content capture may be off in production (D5). Requires Phoenix query API.
Store scoring-relevant content in-platform	Full control. Available regardless of trace backend or content capture setting.	Duplication. Storage cost.

Recommendation: Use AgentOutputEnvelope as primary source + capture judge-relevant content at scoring time.

The envelope already has `result` (the agent's structured output), `reasoning_summary`, `reasoning_chain`, and the step's `stepInput` (via WorkflowAgentContext). For most scoring criteria (correctness, completeness, relevance), this is sufficient — the judge evaluates "given this input, is this output good?" rather than inspecting raw LLM token-level conversation.

For criteria that need the raw prompt/completion (e.g., safety screening of what the LLM actually said), the scoring pipeline should pull from the trace store when `MEDIFORCE_OTEL_CAPTURE_CONTENT=true` and fail gracefully when it's not. This is an advanced path — not Layer 2 MVP.

3D. LAYER 3 PREREQUISITES: WHAT LAYER 2 MUST CAPTURE

For "add this agent run to an eval dataset" (Layer 3) to work, the Score entity must carry or be joinable to:

DATA NEEDED FOR DATASET ENTRY	WHERE IT LIVES	LAYER 2 ACTION
Input (what the agent was asked)	`StepExecution.input` / `WorkflowAgentContext.stepInput`	Score has `agentRunId` → join to AgentRun → join to StepExecution
Output (what the agent produced)	`AgentOutputEnvelope` on AgentRun	Same join path
Expected output (ground truth)	Doesn't exist today	Layer 3 concern — but the human verdict Score IS implicit ground truth
Model used	`AgentOutputEnvelope.model`	Already on AgentRun
Quality judgment	Score.value + Score.label	This IS the Score
Correlation context	namespace, workflow name+version, stepId	Score carries these via correlation fields

Key insight: A human verdict Score (source='human', name='human_verdict') on an AgentRun with `intent='success'` is an implicit "this output is correct" signal. Layer 3 can offer: "This agent run was approved by a human. Add it to the eval dataset as a golden case?" The Score doesn't need to store the input/output — it just needs the join key (`agentRunId`) to retrieve them.

Score must carry enough correlation to support Layer 3-4 queries:

- `agentRunId` (the subject)
- `processInstanceId` (which workflow run)
- `namespace` (scoping)
- `stepId` (which step in the workflow)
- Workflow `name` + `version` (via the AgentRun → StepExecution join, or denormalized on Score for query performance)

Proposed Score Schema (sketch)

Based on the research above, a minimal Score schema for Layer 2:

```
const ScoreSchema = z.object({
  id: z.string().uuid(),

  // What is being scored
  subject: z.discriminatedUnion('type', [
    z.object({ type: z.literal('agent_run'), id: z.string() }),
    z.object({ type: z.literal('workflow_run'), id: z.string() }),
  ]),

  // The score itself
  name: z.string().min(1),              // e.g. 'human_verdict', 'llm_judge', 'has_result'
  value: z.number().min(0).max(1),      // normalized 0–1
  label: z.string().nullable(),         // categorical label, e.g. 'approve', 'reject'
  comment: z.string().nullable(),       // reasoning / explanation

  // Source discrimination
  source: z.enum(['human', 'llm_judge', 'deterministic']),

  // Who/what created this score
  createdBy: z.string().nullable(),     // userId for human, null for automated

  // Metadata (extensible, source-specific)
  metadata: z.record(z.string(), z.unknown()).nullable(),
  // For llm_judge: { judgeModel, confidence, rubric, ... }
  // For deterministic: { rule, threshold, actual, ... }
  // For human: { verdictKey, intent, ... }

  // Correlation (denormalized for query performance)
  namespace: z.string(),
  processInstanceId: z.string().nullable(), // null if scoring a standalone agent run
  stepId: z.string().nullable(),

  // Immutability
  supersedes: z.string().uuid().nullable(), // points to previous Score if revised

  // Timestamps
  createdAt: z.string().datetime(),

  // Optional: Score Config reference (Layer 2 stretch)
  configId: z.string().uuid().nullable(),
});
```

This follows the patterns from existing platform entities: Zod schema, namespace-scoped, correlation fields for joins, append-only with `supersedes` for revisions.

Open Questions for the Team

DESIGN DECISIONS

1. **Should deterministic scores auto-fire on every agent run, or be opt-in per workflow step?** Auto-fire is simpler and gives universal baseline coverage. Opt-in reduces noise for workflows where certain checks don't apply. Proposal: auto-fire a small set (has_result, confidence_threshold) always; additional checks opt-in via step config.

2. **Should LLM-as-judge scores block the workflow or run async?** Blocking adds latency and cost to every agent step. Async means the score arrives after the step has already progressed. For L3 (human review required), the human is already blocking — running the LLM judge while waiting for the human is free latency-wise. For autonomous steps (L0-L2, L4), async is the only sane option.

3. **Score Config entity — Layer 2 or defer?** A Score Config would define "how to score" (rubric, judge model, threshold, enabled/disabled) and be attached to a workflow step definition. This makes scoring declarative and configurable. But it's also a new entity, new UI, new handler. Could ship Layer 2 with hardcoded score names and add Config in Layer 2.5.

4. **Denormalize workflow name+version on Score, or always join through AgentRun?** Denormalization speeds up "all scores for workflow X" queries but adds write-time complexity. The AgentRun already has `processInstanceId` → join to ProcessInstance → has workflow name+version. Propose: start with join, add denormalized columns if query performance demands it.

TECHNICAL DECISIONS

5. **Phoenix sync mechanism.** ~~Resolved~~: Phoenix exposes `POST /v1/span_annotations` as a JSON REST endpoint. TypeScript SDK `@arizeai/phoenix-client` provides typed wrappers generated from OpenAPI. No Python sidecar needed. Remaining question: should the sync be fire-and-forget from the Score creation handler, or a separate background job with retry? Fire-and-forget is simpler but loses scores if Phoenix is down. Background job (BullMQ) is more reliable but adds infra dependency (Redis). Proposal: fire-and-forget for Layer 2 MVP, BullMQ job if we find scores missing in practice.

6. **OTel span ID storage.** Adding `otelSpanId` to AgentRun requires a Postgres migration and a change to `withAgentRunSpan()` in agent-runner.ts. Low-risk, small change. Should it happen as part of Layer 2 or as a standalone prep PR?

7. **Score table design.** Single `scores` table with JSONB `metadata` column (like AuditEvent's pattern), or separate tables per source type? Single table with JSONB is simpler, matches our existing patterns, and the `source` discriminator handles type-specific queries. Propose: single table.

8. **Existing AgentRun fields vs Score.** `AgentOutputEnvelope` already has `confidence` (self-assessment). Should we also auto-create a Score from `confidence`, or keep self-assessment and external judgment as separate concepts? ADR-0007 / CONTEXT.md explicitly distinguishes them: "confidence is the agent's self-assessment, a Score is an external judgment." Propose: do NOT auto-create a Score from confidence. They're different things; mixing them pollutes the score distribution.

SCOPE DECISIONS

9. **Layer 2 MVP scope.** Propose: Score schema + repo + handler (CRUD), auto-create human_verdict Score from L3 approvals, 2-3 deterministic scores auto-firing, basic per-agent score aggregation endpoint. Defer: LLM-as-judge pipeline, Score Config entity, Phoenix sync, dashboard UI, drift alerting.

10. **CLI support.** `mediforce score list --agent-run <id>`, `mediforce score create --agent-run <id> --name <name> --value <value>`. Follows the dogfood rule (CLI > REST). Should land in the same PR as the handler.